

CINEMATCH: AI RECOMMENDATION ENGINE

Complete Mini Project Blueprint & Technical Architecture

Domain: Artificial Intelligence **Tech Stack:** Python, Streamlit, Pandas, NumPy, Scikit-Learn, Surprise

1. Abstract

CineMatch is an AI-powered personalized movie recommendation web application designed to help users discover movies according to their interests and previous rating behavior. The system uses **Collaborative Filtering** techniques to analyze user-movie interactions and predict movies that a user is likely to enjoy.

The project implements both **Memory-Based Filtering using Cosine Similarity** and **Model-Based Filtering using Singular Value Decomposition (SVD)**. The recommendation engine is developed using Python and integrated with an interactive **Streamlit** web interface. The system is designed to provide a simple, efficient, and user-friendly recommendation experience suitable for a B.Tech mini-project demonstration.

2. Problem Statement

With the rapid growth of online streaming platforms, users have access to thousands of movies. However, having too many choices often makes it difficult to decide what to watch.

Traditional movie-search systems generally require users to manually browse through a large collection of movies. The proposed **CineMatch** system addresses this problem by analyzing users' historical ratings and identifying their preferences. Using mathematical and machine-learning techniques, the system predicts ratings for movies that the user has not yet rated and recommends the most relevant movies.

3. Objectives

- Develop an intelligent movie recommendation system based on user preferences and ratings.
- Implement **Memory-Based Collaborative Filtering** using Cosine Similarity.
- Implement **Model-Based Collaborative Filtering** using SVD Matrix Factorization.
- Create a User-Movie **Utility Matrix** for analyzing rating relationships.
- Handle the high sparsity present in movie-rating datasets.
- Evaluate recommendation performance using **RMSE (Root Mean Squared Error)**.
- Generate personalized **Top-N movie recommendations**.
- Develop a simple and interactive Streamlit web interface.
- Keep the project manageable and suitable for a B.Tech mini-project demonstration.

4. Project Scope

- **Dataset Processing:** Load and process the MovieLens dataset containing users, movies, and ratings.
- **Data Analysis:** Perform basic exploratory data analysis to understand rating patterns and movie popularity.
- **Utility Matrix:** Create a User-Movie matrix representing the relationship between users and movies.
- **Similarity Analysis:** Calculate User-User or Item-Item similarity using Cosine Similarity.
- **Matrix Factorization:** Apply SVD to identify hidden/latent preferences from the rating data.
- **Prediction:** Predict ratings for movies that a user has not previously rated.
- **Recommendation:** Rank predicted ratings and generate a personalized Top-5 movie list.
- **Evaluation:** Compare recommendation approaches using RMSE.
- **Web Interface:** Provide a Streamlit interface where users can enter their User ID and receive recommendations.

5. Key Modules & Features

MODULE	PURPOSE & DESCRIPTION
Data Ingestion	Loads the MovieLens dataset containing users, movies, and ratings and prepares the data for processing.
Data Preprocessing	Cleans, merges, and organizes the dataset for recommendation-model development.
Utility Matrix	Converts rating data into a User-Movie matrix and analyzes the sparsity of the dataset.
Memory-Based Model	Uses Cosine Similarity to identify similar users or movies and generate recommendations.
Model-Based Engine	Uses the Surprise SVD algorithm to learn latent features and predict missing movie ratings.
Evaluation & Ranking	Calculates RMSE and ranks predicted movie ratings to generate the Top-N recommendations.
Streamlit Interface	Provides an interactive interface for entering User ID and displaying personalized movie recommendations.

6. Technology Stack

LAYER / AREA	TECHNOLOGY CHOSEN
Frontend Interface	Streamlit
Programming Language	Python
Data Processing	Pandas, NumPy
Machine Learning	Scikit-Learn, Surprise
Recommendation Algorithm	Cosine Similarity, SVD
Data Visualization	Matplotlib, Seaborn
Development Environment	Jupyter Notebook, Visual Studio Code
Dataset	MovieLens Dataset
Evaluation Metric	RMSE

7. Development Roadmap

Phase 1 — Data Preparation: Load the MovieLens dataset, inspect the data, perform preprocessing, merge required datasets, and create the User-Movie Utility Matrix.

Phase 2 — Exploratory Data Analysis: Analyze rating distributions, movie popularity, user activity, and the sparsity of the Utility Matrix.

Phase 3 — Memory-Based Recommendation: Implement User-User and/or Item-Item similarity using Cosine Similarity and generate preliminary recommendations.

Phase 4 — Matrix Factorization: Implement the SVD algorithm using the Surprise library to learn latent features and predict user ratings.

Phase 5 — Evaluation: Evaluate the recommendation models using RMSE and compare the performance of the implemented approaches.

Phase 6 — Recommendation & Ranking: Develop the Top-N recommendation function that sorts predicted ratings and selects the highest-rated movies for a particular user.

Phase 7 — Streamlit Integration: Integrate the trained recommendation engine with Streamlit and create an interactive interface for real-time recommendations.

Phase 8 — Testing & Documentation: Test different User IDs, verify recommendation results, prepare screenshots, diagrams, project documentation, and final presentation.

8. Basic System Flow

User Enters ID → System Loads User Profile → Checks Previously Rated Movies → SVD Predicts Ratings for Unrated Movies → Sorts Predicted Ratings → Selects Top 5 Movies → Displays Personalized Recommendations

9. Expected Outcomes

- A working **AI-based personalized movie recommendation web application**.
- A functional recommendation engine capable of predicting user preferences.
- Implementation of both **Cosine Similarity** and **SVD Matrix Factorization** approaches.
- A comparison of recommendation models using RMSE.
- An interactive Streamlit interface for entering User ID and viewing recommendations.
- Personalized Top-5 movie recommendations based on predicted ratings.
- A complete project structure suitable for B.Tech mini-project review, demonstration, and documentation.

10. Future Scope

- **Hybrid Recommendation:** Combine Collaborative Filtering with Content-Based Filtering using movie genres, tags, actors, directors, etc.
- **Cold Start Handling:** Develop popularity-based or content-based recommendations for new users with no rating history.
- **Deep Learning:** Replace traditional SVD with Neural Collaborative Filtering using TensorFlow or PyTorch.
- **User Authentication:** Add login/signup functionality and maintain individual user profiles.
- **Movie Search:** Add movie search and filtering features.
- **Advanced Recommendations:** Provide personalized recommendations based on genres, ratings, watch history, and user preferences.
- **Deployment:** Deploy the application online so that users can access the recommendation system through a web browser.

11. Suggested Team Responsibilities

TEAM MEMBER	CORE RESPONSIBILITIES
Ayush Rajput	End-to-end project development, data preprocessing, EDA, Utility Matrix creation, Cosine Similarity implementation, SVD model development, RMSE evaluation, recommendation logic, Streamlit UI, testing, documentation, and final presentation.

12. Recommended Project Approach

Develop the project in stages, beginning with the **MovieLens dataset and data preprocessing**. First create the Utility Matrix and implement the basic similarity-based recommendation approach. After that, implement the **SVD Matrix Factorization model** and evaluate its performance using RMSE.

Once the recommendation engine is working correctly, integrate it with the **Streamlit interface**. Finally, test the system with different User IDs and prepare the project report, screenshots, diagrams, presentation, and final demonstration.

This approach keeps **CineMatch** manageable for a B.Tech mini-project while still demonstrating important concepts of **Artificial Intelligence, Machine Learning, Collaborative Filtering, Matrix Factorization, and Web Application Development**.